

Project Kickstart

AI-driven playbook for project initiation — **10 steps to launch a clean, governed, build-ready code repository**

After Step-00, each step is one Claude command, run in a fresh session **from the kit**. It drafts the step's documents in your **target repo** and opens a PR/MR. The documents become final only when the Product Owner and/or Architect approves and merges it to `main`.

KIT · run commands Control plane Commands, step guides, templates, step registry, role files.	TARGET · registered by Step-00 Your project clone Every git operation and every produced document lands here.	GIT HOST · GitHub or others Review and merge One branch and one PR/MR per step. Merge to <code>main</code> = final.
---	---	---

What each step produces and waits for

Every step's document moves through the same four states. On a branch it's a draft; on `main` it's final.

HUMAN · starts Initiate Run from the kit; nothing in the target yet <code>/proj-init-bootstrap</code>	CLAUDE · drafts Draft On <code>init/<step></code> in the target <code>/proj-init-<step></code>	HUMAN · approves Review Pushed after your yes, PR/MR open <code>gh pr create</code>	HUMAN · merges Final Merged after the self-review checklist <code>merge to main</code>
--	---	--	---

A step won't start until its inputs below are on `main`; steps 02–09 also need `CONTRIBUTING.md`.

Step	Document	Purpose	Needs on main
01	<code>CONTRIBUTING.md</code>	Governance and tooling rules	Step-00 registration
02	<code>PRODUCT.md</code>	Product concept	<code>CONTRIBUTING.md</code>
03	<code>PRD.md</code>	Product requirements	<code>PRODUCT.md</code>
04	<code>ARCHITECTURE.md</code>	System design	<code>PRD.md</code>
05	<code>TECH-STACK.md</code>	Approved technologies	<code>PRD.md</code> , <code>ARCHITECTURE.md</code>
06	<code>ENGINEERING-RULES.md</code> + <code>CLAUDE.md</code>	Coding rules; AI assistant instructions	<code>ARCHITECTURE.md</code> , <code>TECH-STACK.md</code>
07	<code>README.md</code>	Project entry point	All documents from 02–06
08	<code>BACKLOG.md</code>	Initial backlog	02–06 + <code>README.md</code>
09	<code>CONTRIBUTING.md</code>	Permanent contribution core	<code>BACKLOG.md</code>

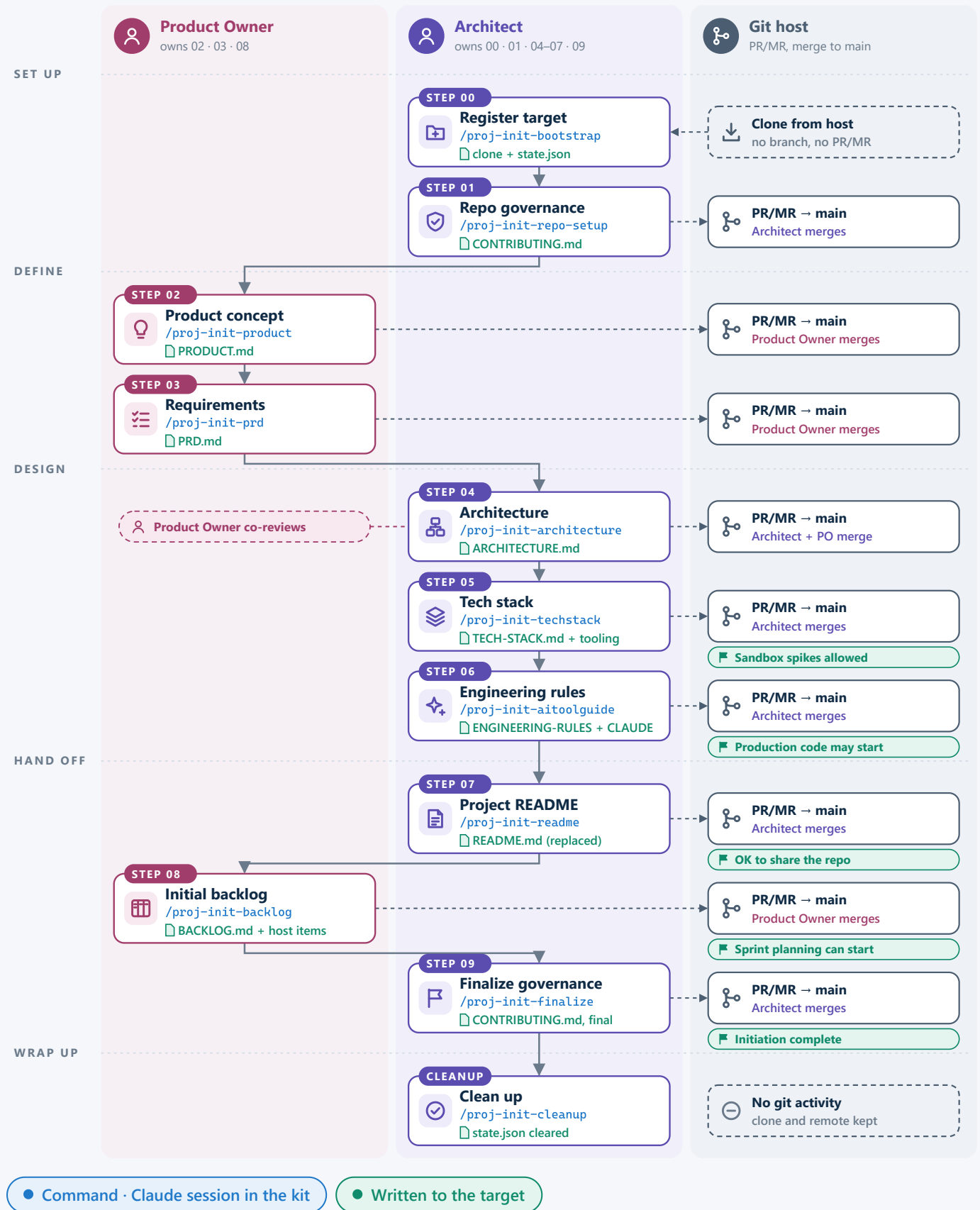
Start every step the same way

TERMINAL	<code>cd <kit-folder></code> <code>claude</code>	# from the kit folder – never from the target clone
CLAUDE	<code>/proj-init-<step></code>	# e.g. <code>/proj-init-product</code>

One step, one session. For the next step, run `/clear` or start a new `claude` session. Each step syncs `main` and checks status itself; run `/proj-init-doc-status` any time to see what's next.

The whole flow, by who does what

Read top to bottom. Each step sits in the lane of the role that owns it; an elbow in the arrow is a handoff. The right column shows each PR/MR, who merges it, and what that merge unlocks.



Docs land under `docs/` in the target; `README.md` and `CONTRIBUTING.md` stay at the root. Solo team? Wear the hat of the lane the step sits in.

Commands overview

Every command runs from the kit folder in a fresh Claude session. Each step is covered in detail on the pages that follow.

INITIATION · RUN IN ORDER

STEP 00 /proj-init-bootstrap

Register target

Clones the project repository from your Git host into an empty or new local folder and registers that clone as the initiation workspace. The command presents a dry run for review and applies it only after you confirm. From then on, every command runs from the kit but operates on the registered clone, so kit files are never copied into the project.

PRODUCES The local clone, plus `.proj-init/state.json` in the kit (gitignored). Every later step and utility reads this file to locate the target before doing any git work. It is specific to one machine; to continue elsewhere, re-run Step-00.

STEP 01 /proj-init-repo-setup

Repo governance

Establishes the repository's governance baseline: the branching convention, the commit message convention, and the self-review checklist every PR/MR must satisfy. Where the Git host and plan permit, it also enables branch protection on `main`. Rules needed only during initiation are placed in a fenced `INITIATION-ONLY` block so they can be removed cleanly at the end.

PRODUCES `CONTRIBUTING.md`. Steps 02–09 will not start until this file is on `main`. Its permanent rules govern every contribution for the life of the project; Step-05 extends it and Step-09 finalizes it.

STEP 02 /proj-init-product

Product concept

Conducts a structured interview with the Product Owner, one question at a time, to define the product's vision, problem statement, objectives, target users, features, scope boundaries, and success criteria. No answer is assumed; each section is drafted and revised until it is approved.

PRODUCES `docs/PRODUCT.md`. The source of truth for product intent: requirements (03) are derived from it, and the README (07) and backlog (08) draw on it. Later changes to scope or goals go through `/proj-init-doc-update`.

STEP 03 /proj-init-prd

Requirements

Translates the approved product concept into a formal requirements document: user stories, functional and non-functional requirements, acceptance criteria, out-of-scope items, dependencies, constraints, and risks. Every requirement is stated so that it can be tested.

PRODUCES `docs/PRD.md`. Architecture (04) and technology (05) decisions are made against these requirements, and the backlog's epics and stories (08) are derived from them. Its acceptance criteria give the team a testable definition of each feature.

STEP 04 /proj-init-architecture

Architecture

Defines the system architecture: components and their boundaries, data design, data flow, integration points, security posture and data classification, and the non-functional approach. It records the key design decisions and the implementation conventions code must follow. The session loads both Architect and Product Owner context, and both must approve.

PRODUCES `docs/ARCHITECTURE.md`. The structural boundary for all implementation. Technology choices (05) and engineering rules (06) are derived from it; design changes after initiation go through `/proj-init-doc-update`.

STEP 05 `/proj-init-techstack`

Tech stack

Selects and locks the approved technologies (languages and frameworks, datastores, cloud and infrastructure services, key libraries and tools) with the trade-offs behind each choice and their versions and constraints. It also adds the project's build, test, and lint commands to `CONTRIBUTING.md` as a tooling layer, leaving the Step-01 governance rules untouched.

PRODUCES `docs/TECH-STACK.md` + tooling layer in `CONTRIBUTING.md`. Only technologies listed here may be used; adding or replacing one requires `/proj-init-doc-update`. Once merged, isolated sandbox spikes may begin.

STEP 06 `/proj-init-aitoolguide`

Engineering rules

Defines the engineering rules every contributor follows, human or AI: coding conventions, scope boundaries, banned patterns, testing and documentation rules, decision-escalation criteria, off-limits files, and workflow conventions. The rules are derived from the approved architecture and tech stack. `CLAUDE.md` points to the shared rules and adds only Claude Code-specific instructions.

PRODUCES `docs/ENGINEERING-RULES.md` + `CLAUDE.md`. Production implementation may start once this is merged. Claude Code reads `CLAUDE.md` automatically in every session, so AI-assisted work follows the same rules as the team.

STEP 07 `/proj-init-readme`

Project README

Replaces the repository's boilerplate README with the project's real entry point: overview, key concepts, prerequisites, environment setup, install and run, tests, project structure, and links to the source-of-truth documents. Every command in it is executed and verified before commit, and no placeholder may remain.

PRODUCES `README.md` (replaced). The first document a new contributor reads and the basis for onboarding. It must be merged before the repository is shared more widely.

STEP 08 `/proj-init-backlog`

Initial backlog

Derives the initial epics and stories from the product and requirements documents and presents them for approval before anything is written. Once approved, it creates the work items in the host tracker one at a time through `gh` or `az boards`; if neither CLI is available, it records `PENDING` IDs and manual creation notes.

PRODUCES `docs/BACKLOG.md` + host work items. The starting point for sprint planning, which may begin once it is merged. Its presence on `main` is also the gate for Step-09.

STEP 09 `/proj-init-finalize`

Finalize governance

Converts `CONTRIBUTING.md` from its initiation form into its permanent form. It removes the `INITIATION-ONLY` block, markers included, and inserts a `DEVELOPMENT-PHASE-GOVERNANCE` stub; the permanent rules and the tooling layer remain verbatim. It halts if the initiation markers are missing.

PRODUCES `CONTRIBUTING.md` (final). The permanent contribution policy for the development phase. Merging it completes initiation.

CLEANUP /proj-init-cleanup

Clean up

Closes out the initiation workspace. It verifies that `BACKLOG.md` is on `main` and the `INITIATION-ONLY` block is gone, then, after your confirmation, unregisters the workspace. The target clone and its remote are not modified.

PRODUCES Deletes `.proj-init/state.json`; nothing in the target. Frees the kit to register the next project; the finished project stays reachable by the utilities below with `--target`.

AFTER INITIATION · ANY TIME

UTILITY /proj-init-doc-status

Status

Produces a read-only status report: which steps are merged to `main`, which have open PRs/MRs, and which step comes next. With `--target <folder>`, it reports on a finished or different project without re-registering it.

PRODUCES A status report in the session; writes nothing. Every step runs this check before it starts; run it yourself to see where a project stands at any time.

UTILITY /proj-init-doc-update <doc>

Update a document

Updates a finalized source-of-truth document on a `docs/update/<doc>` branch, through the same review gate that originally approved it. It analyzes the impact of the change and flags the downstream documents that may need to follow.

PRODUCES The updated document and a reconciliation checklist, opened as a PR/MR. Keeps the documentation aligned as the project evolves; work through the checklist afterward with one update per flagged document.

KIT MAINTENANCE

KIT /proj-init-audit

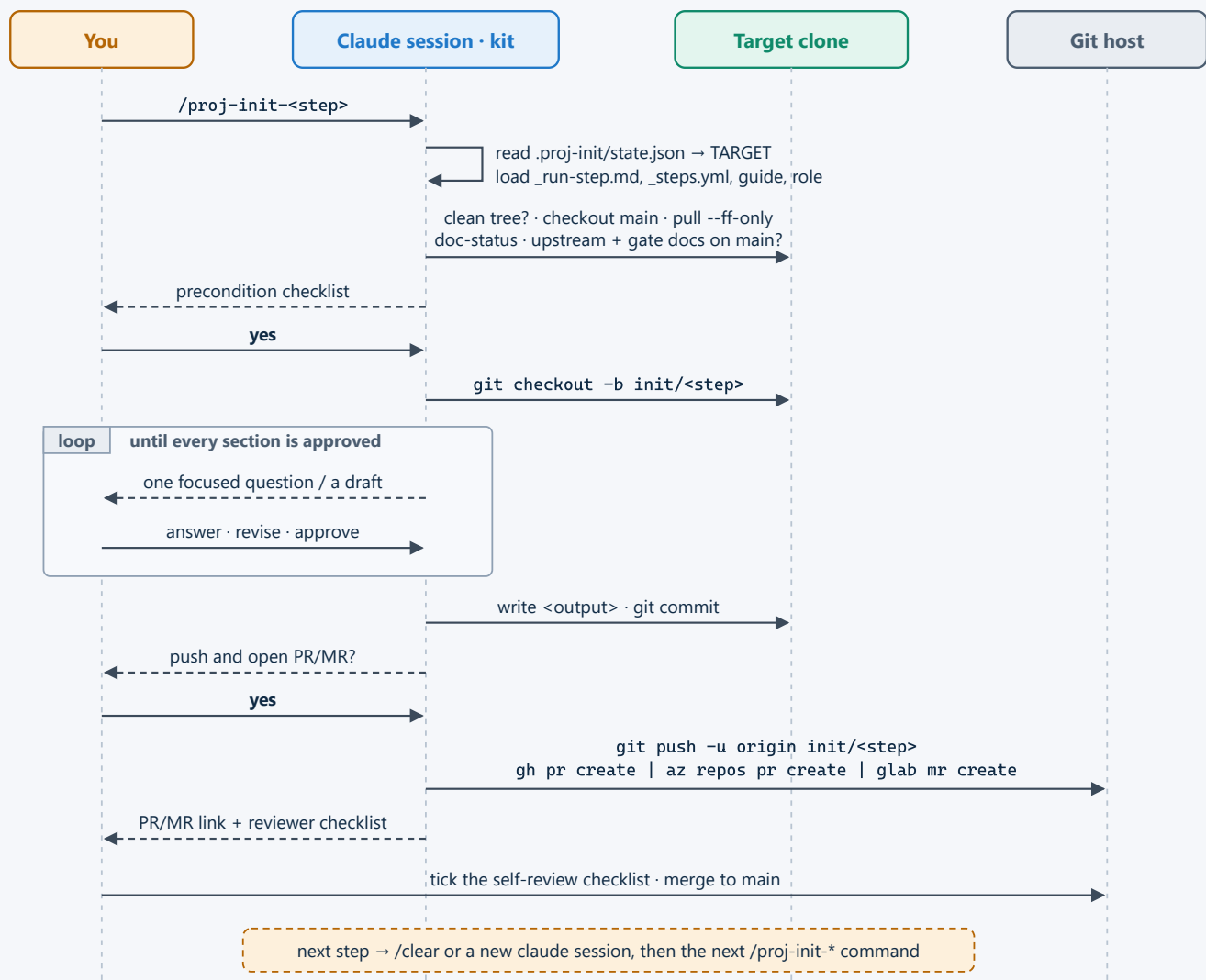
Kit audit

Performs a static completeness audit of the kit itself, scoring dimensions such as control-plane execution, step outputs, resumability, question-driven drafting, and adapter parity. It reads kit files only: it never touches a target, runs a script, or invokes git.

PRODUCES A scored PASS / GAP / FAIL report; writes nothing. Intended for kit maintainers, to catch gaps and drift between the guides, the step registry, and the adapters before teams use the kit.

Anatomy of a step

What happens between you, the Claude session in the kit, the target clone, and your Git host during any step from 01 to 09.



After the PR/MR opens

The command stops once the PR/MR is open. The rest is yours.

1 · WAIT Fresh eyes	Leave the PR/MR at least a few hours, ideally a full day, before you review it. This is the first item on the self-review checklist.
2 · REVIEW Self-review checklist	Tick the rest in the PR/MR description: every section the step guide requires is covered, no upstream doc changed after you branched, and the change came through a PR/MR, not a direct push.
3 · MERGE Merge = final	Merging to <code>main</code> is what makes the document final; there are no draft flags. Step-04 needs both the Architect and the Product Owner to approve.
4 · CONTINUE Next step	Run <code>/clear</code> or start a new <code>claude</code> session, then the next command. It fast-forwards <code>main</code> in the target and stops if an upstream doc isn't merged yet.

Step by step

Run every command from the kit folder, one per fresh session. Steps 01–09 share one runner: resolve the target, fast-forward `main`, check status and preconditions, wait for your **yes**, branch, interview you one question at a time, draft, commit, then push and open the PR/MR only after a second **yes**.

Each step card is tagged with where it runs and who acts:

- Kit** Runs in the kit repo. You start each Claude command here, and the guides and templates load from here.
- Target** Lands in your project clone: the step's branch, commits, and documents.
- Human** Needs a person to act: confirm, review, approve, or merge. The step's owner (Product Owner and/or Architect) does this.

00

Register target

Kit

Human

Clones the project repository from your Git host and registers that clone as the initiation workspace. From then on, every command runs from the kit but operates on the registered clone.

CLAUDE	/proj-init-bootstrap
ASKS	The target's git URL, and an empty or non-existent local folder to clone into.
RUNS	<code>node scripts/bootstrap-target-repo.mjs --target <folder> --url <git-url></code> as a dry run, then again with <code>--apply</code>
WRITES	The clone at <code><folder></code> , plus <code>.proj-init/state.json</code> in the kit (gitignored, local to this machine)
GATE	■ You review the dry run before <code>--apply</code> . Only one workspace at a time; replace it only with <code>--force</code> .
GIT	No branch, no commit, no PR/MR.

01

Repo governance

Kit

Target

Human

Establishes the repository's governance baseline: the branching convention, the commit message convention, and the self-review checklist every PR/MR must satisfy.

CLAUDE	/proj-init-repo-setup
OWNER	Architect · reviewed by Architect
WRITES	<code>CONTRIBUTING.md</code> : permanent rules plus an <code>INITIATION-ONLY</code> fenced block, and branch protection on <code>main</code> if your host and plan allow it
GIT	<code>init/repo-setup</code> · <code>docs: add repo governance baseline (Step-01)</code>
GATE	■ Self-review checklist, then you merge. Every later step stops until <code>CONTRIBUTING.md</code> is on <code>main</code> .

Product concept

● Kit

● Target

● Human

Conducts a structured interview with the Product Owner, one question at a time, to define the product's vision, problem statement, objectives, target users, features, scope boundaries, and success criteria.

CLAUDE

/proj-init-product

OWNER

Product Owner · reviewed by Product Owner

READS

CONTRIBUTING.md on main (the Step-01 gate)

WRITES

docs/PRODUCT.md

GIT

init/product · docs: add PRODUCT.md (Step-02)

GATE

■ The Product Owner approves and merges.

03

Requirements

● Kit

● Target

● Human

Translates the approved product concept into a formal requirements document. Every requirement is stated so that it can be tested.

CLAUDE

/proj-init-prd

OWNER Product Owner · reviewed by Product Owner

READS docs/PRODUCT.md

WRITES docs/PRD.md

GIT init/prd · docs: add PRD.md (Step-03)

GATE ■ The Product Owner approves and merges.

04

Architecture

● Kit

● Target

● Human

Defines the system architecture and records the key design decisions and the implementation conventions code must follow. The session loads both Architect and Product Owner context.

CLAUDE

/proj-init-architecture

OWNER Architect · reviewed by Architect + Product Owner

READS docs/PRD.md

WRITES docs/ARCHITECTURE.md

GIT init/architecture · docs: add ARCHITECTURE.md (Step-04)

GATE ■ The Architect and Product Owner both approve, then merge.

05

Tech stack

● Kit

● Target

● Human

Selects and locks the approved technologies with the trade-offs behind each choice, and adds the project's build, test, and lint commands to CONTRIBUTING.md as a tooling layer.

CLAUDE

/proj-init-techstack

OWNER Architect · reviewed by Architect

READS docs/PRD.md, docs/ARCHITECTURE.md

WRITES docs/TECH-STACK.md ; appends the tooling layer to CONTRIBUTING.md and leaves the Step-01 governance layer intact

GIT init/techstack · docs: add TECH-STACK.md and CONTRIBUTING tooling layer (Step-05)

GATE ■ The Architect approves and merges.

AFTER MERGE Sandbox spikes are allowed, on isolated spike branches that never merge to main.

06

Engineering rules

● Kit

● Target

● Human

Defines the engineering rules every contributor follows, human or AI, derived from the approved architecture and tech stack. `CLAUDE.md` points to the shared rules and adds only Claude Code-specific instructions.

CLAUDE

/proj-init-aitoolguide

OWNER

Architect · reviewed by Architect

READS

`docs/ARCHITECTURE.md` , `docs/TECH-STACK.md`

WRITES

`docs/ENGINEERING-RULES.md` and `CLAUDE.md`

GIT

`init/aitoolguide` · docs: add `ENGINEERING-RULES.md` and `CLAUDE.md` (Step-06)

GATE

■ The Architect approves and merges.

AFTER MERGE

Production implementation can start.

07

Project README

● Kit

● Target

● Human

Replaces the repository's boilerplate README with the project's real entry point: overview, prerequisites, environment setup, install and run, tests, and links to the source-of-truth documents.

CLAUDE

/proj-init-readme

OWNER

Architect · reviewed by Architect

READS

`PRODUCT.md` , `PRD.md` , `ARCHITECTURE.md` , `TECH-STACK.md` , `ENGINEERING-RULES.md`

WRITES

`README.md` , replacing the existing file with all placeholders removed

GIT

`init/readme` · docs: replace boilerplate README with project README (Step-07)

GATE

■ Every command in the README is executed and verified before commit. The Architect approves and merges.

AFTER MERGE

Required before the repo is shared more widely.

08

Initial backlog

● Kit

● Target

● Host

● Human

Derives the initial epics and stories from the product and requirements documents and creates them as work items in the host tracker.

CLAUDE

/proj-init-backlog

OWNER

Product Owner · reviewed by Product Owner

READS

The five docs Step-07 reads, plus `README.md`

WRITES

`docs/BACKLOG.md` , plus host work items via `gh` or `az boards` ; with neither CLI available, `PENDING` IDs and manual creation notes

GIT

`init/backlog` · docs: add `BACKLOG.md` – Step-08 initial backlog

GATE

■ You approve the derived backlog before anything is written to the host or to disk. Items are then created one at a time. The Product Owner merges.

AFTER MERGE

Required before sprint planning. Next: Step-09.

09

Finalize governance

● Kit

● Target

● Human

Converts `CONTRIBUTING.md` from its initiation form into its permanent form: the permanent contribution policy for the development phase.

CLAUDE

/proj-init-finalize

OWNER

Architect · reviewed by Architect

READS

`docs/BACKLOG.md` on `main` (the gate) and `CONTRIBUTING.md` on `main`

WRITES

`CONTRIBUTING.md`: the `INITIATION-ONLY` block is removed (markers included) and a `DEVELOPMENT-PHASE-GOVERNANCE` stub is inserted. Permanent rules and the tooling layer stay verbatim.

GIT

`init/finalize` · `docs: finalize CONTRIBUTING.md for the development phase (Step-09)`

GATE

■ Stops if the `INITIATION-ONLY` markers are missing. Once the Architect merges, initiation is complete.

✓

Clean up

● Kit

● Human

Closes out the initiation workspace by unregistering it. The target clone and its remote are not modified.

CLAUDE

/proj-init-cleanup

CHECKS

`docs/BACKLOG.md` is on `main` and `INITIATION-ONLY` is gone from `CONTRIBUTING.md`

RUNS

`node scripts/bootstrap-target-repo.mjs --clear`, after your **yes**

WRITES

Deletes `.proj-init/state.json`. Nothing else.

GIT

None.

Utilities, any time

CLAUDE

```
/proj-init-doc-status           # read-only: merged, open PRs/MRs, what's next
/proj-init-doc-update <doc>    # e.g. PRD.md → PR/MR + reconciliation checklist
# add --target <folder> to either for a finished or other project
```

Which document to update

If this happens	Run <code>/proj-init-doc-update</code> on
Product scope, goal, or target users change	<code>PRODUCT.md</code>
Requirements or acceptance criteria change	<code>PRD.md</code>
System design or a major architecture decision changes	<code>ARCHITECTURE.md</code>
A library, framework, or tool is added, removed, or version-locked	<code>TECH-STACK.md</code>
Coding conventions, banned patterns, or testing rules change	<code>ENGINEERING-RULES.md</code>
Setup steps, env config, or run commands change	<code>README.md</code>
Epics or stories added, removed, rescoped, or host IDs change	<code>BACKLOG.md</code>

Rules that hold at every step

One step, one fresh session. Never run two step commands in the same session. Status is checked first, then only the step you asked for runs.	Run from the kit, write to the target. Commands load guides from the kit and do git work in the registered clone. Kit files are never copied in.
Merge = final. A doc on a branch is a draft; a doc on <code>main</code> is final. There are no draft files and no status flags.	Nothing ships without your yes. Explicit confirmation before branching, before every host write, and before every push.
Never on <code>main</code>. No commits or pushes to <code>main</code> , and no force-pushes to shared branches. Every change goes through a PR/MR.	Upstream first. A step stops if any doc it depends on isn't on <code>main</code> , or if its own output is already final (use <code>/proj-init-doc-update</code> instead).
Same machine, same operator. <code>.proj-init/state.json</code> is local to one machine. To continue elsewhere, re-run Step-00.	Never assume an answer. Claude asks one focused question at a time until no template placeholder is left.

Milestones

AFTER 05 MERGES Spikes Isolated spike branches only, never merged to <code>main</code> .	AFTER 06 MERGES Production code Implementation work can begin.	AFTER 07 MERGES Share the repo The README is the real entry point.	AFTER 08 MERGES Sprint planning Then Step-09 completes initiation.
--	--	--	--

Solo / process-enforced: one person may wear both hats; take on the one each step calls for. Source of truth: docs/guides/proj-init/(_steps.yml, _run-step.md, step guides).